PIYUSH KUMAR SINGH

25BAI11043

COMPUTER SCIENCE AND ENGINERRING (AIML)

2025-2029 BATCH

AIML PROJECT REPORT

MUSIC MOOD RECOMMENDATION SYSTEM

29TH MARCH 2026

- ***Abstract***

This report presents the design, implementation, and evaluation of a Music Mood Recommendation System developed as part of the AIML Vityarthi Build Your Own Project (BYOP) initiative. The system leverages supervised machine learning techniques to classify songs into mood categories and recommend tracks that align with a user's selected mood.

The project utilises a dataset of 113,549 Spotify tracks, each characterised by nine audio features including valence, energy, danceability, acousticness, loudness, tempo, instrumentalness, speechiness, and liveness. A Random Forest classifier is trained to predict mood labels — Happy, Sad, Angry, and Calm — achieving a test accuracy of 99.1% and a precision and recall above 98% across all mood categories.

The system is accessible via a command-line interface that allows users to obtain mood-based song recommendations interactively or programmatically. The codebase follows software engineering best practices including centralised configuration, shared utility modules, and a complete pytest test suite.

Table of Contents

- *Abstract* 1
- *Introduction* 3
- *Objectives* 3
- *System Architecture and Project Structure* 4
- *Dataset Description* 5
- *Methodology* 6
  - *Data Preprocessing* 6
  - *Feature Engineering and Mood Labelling* 7
  - *Model Training* 7
  - *Recommendation Engine* 8
  - *Technologies and Libraries* 8
  - *Configuration Management* 9
  - *Utility Modules* 10
- *Model Performance and Evaluation* 10
- *Results* 12
- *Challenges and Solutions* 13
- *Future Enhancements* 13
- *Conclusion* 14
- *References* 14

• Introduction

Music plays a profound role in human emotional experience. People instinctively reach for different music depending on their mood — upbeat tracks when energised, mellow tunes when reflective, and calming melodies when stressed. Despite this intuitive connection, most music streaming platforms rely on collaborative filtering or listening history to recommend songs, which fails to capture the user's current emotional state.

This project addresses that gap by building a mood-aware music recommendation system. By analysing the acoustic properties of songs — features that Spotify computes for every track in its catalogue — the system predicts the mood a song is likely to evoke and uses that prediction to surface relevant recommendations.

The system is built end-to-end: from raw data ingestion and mood labelling, through model training and evaluation, to a fully functional command-line interface. All components are modular, testable, and configurable, demonstrating sound software engineering alongside machine learning competency.

• Objectives

- Design and implement a complete end-to-end music recommendation pipeline.
- Assign mood labels to songs using rule-based heuristics on Spotify audio features.
- Train a supervised Random Forest classifier to predict song mood from audio features.

- Achieve high classification accuracy (target: above 95%) across all mood categories.
- Build a CLI-based recommendation interface supporting both interactive and argument-driven usage.
- Apply software engineering best practices: centralised config, utility modules, and automated testing.
- Maintain a clean, well-documented GitHub repository with a professional README.

System Architecture and Project Structure

The project is organised as a modular Python application with clearly separated concerns across data processing, model training, recommendation, configuration, and testing layers.

Component	File / Location	Responsibility
Data Preprocessing	src/data_preprocessing.py	Load, clean, and label raw track data
Model Training	src/model_training.py	Train, evaluate, and persist the classifier
Recommendation Engine	src/recommendation.py	Filter and return mood-matched songs
CLI Entrypoint	main.py	Argument parsing and user interaction
Lightweight Trainer	scripts/train_models.py	Quick training script for testing
Configuration	config.py	Centralised paths and hyperparameters

Utilities	utils.py	Logging, model save/load helpers
Test Suite	tests/	Unit tests for all three core modules

<u>The directory structure is as follows:</u>

```
aiml-project/
  data/raw/dataset/tracks.csv        Raw Spotify track data
  data/processed/music_with_mood.csv  Cleaned and labelled data
  models/mood_model.pkl              Trained Random Forest
  models/label_encoder.pkl           Mood label encoder
  src/  data_preprocessing.py  model_training.py  recommendation.py
  tests/  test_preprocessing.py  test_model.py  test_recommendation.py
  config.py  main.py  utils.py  requirements.txt  pytest.ini
```

• ***Dataset Description***

The dataset used in this project is a Spotify Tracks Dataset sourced from Kaggle, containing 113,549 individual track records spanning a wide variety of genres. Each record includes audio features computed by Spotify's audio analysis API.

Attribute	Description	Range

danceability	How suitable a track is for dancing	0.0 to 1.0
energy	Perceptual measure of intensity and activity	0.0 to 1.0
loudness	Overall loudness of the track in decibels	-60 to 0 dB
speechiness	Presence of spoken words in the track	0.0 to 1.0
acousticness	Confidence measure of acoustic sound	0.0 to 1.0
instrumentalness	Ratio of instrumental to vocal content	0.0 to 1.0
liveness	Presence of live audience in the recording	0.0 to 1.0
valence	Musical positiveness conveyed by the track	0.0 to 1.0
tempo	Estimated tempo in beats per minute	0 to 250 BPM
mood	Derived target label (Happy/Sad/Angry/Calm)	4 categories

The dataset required no external mood labels — mood categories were derived from the audio features themselves using domain-driven heuristics, as described in Section 5.2.

• ***Methodology***

• ***Data Preprocessing***

The raw dataset is loaded from tracks.csv and subjected to the following cleaning operations:

- Removal of records with missing values in any of the nine feature columns or the target column.
- Deduplication based on track name to avoid model bias.
- Feature normalisation where applicable.
- Index reset post-cleaning for consistent downstream alignment.

After preprocessing, the cleaned data is persisted to data/processed/music_with_mood.csv for use during model training.

- ## Feature Engineering and Mood Labelling

Since the original dataset does not include mood annotations, mood labels are derived algorithmically from the audio features using threshold-based heuristics informed by music psychology research:

Mood	Primary Indicators	Characteristic Values
Happy	High valence, high energy, high danceability	valence > 0.6, energy > 0.6
Sad	Low valence, low energy, high acousticness	valence < 0.4, energy < 0.4
Angry	High energy, low valence, high loudness	energy > 0.7, valence < 0.35
Calm	Low energy, moderate valence, high acousticness	energy < 0.4, acousticness > 0.5

- ## Model Training

The labelled dataset is split into training (80%) and testing (20%) subsets using a stratified split to maintain class distribution. A Random Forest classifier is trained on the nine audio feature columns.

Random Forest was selected for this task due to its robustness to noisy features, its built-in handling of non-linear decision boundaries, and its resistance to overfitting through ensemble averaging. The key hyperparameters are configured centrally via config.py:

- n_estimators: 100 trees
- random_state: 42 (for reproducibility)
- test_size: 0.2

The trained model and label encoder are serialised to disk as .pkl files using Python's pickle module, enabling fast loading during inference without retraining.

- ***Recommendation Engine***

The recommendation engine operates as follows:

- The trained model and processed dataset are loaded from disk.
- The user specifies a mood via CLI argument or interactive prompt.
- The dataset is filtered to tracks whose predicted mood matches the requested category.
- A random sample of top-K tracks is returned, displaying track name, artist, and genre.

The top-K value defaults to the value set in config.py (TOP_N_RECOMMENDATIONS) but can be overridden via the --top-k CLI argument.

- ***Implementation Details***

- ***Technologies and Libraries***

Library	Version	Purpose
Python	3.7+	Core programming language
pandas	>=1.3.0	Data loading, cleaning, and manipulation
numpy	>=1.21.0	Numerical operations
scikit-learn	>=1.0.0	Random Forest classifier and evaluation metrics
matplotlib	>=3.5.0	Data visualisation and EDA
seaborn	>=0.11.0	Statistical visualisation
pytest	>=7.0.0	Automated unit testing
pytest-cov	>=4.0.0	Test coverage reporting

- ***Configuration Management***

All file paths, model parameters, and recommendation settings are centralised in config.py. This eliminates hardcoded values across the codebase and enables easy adjustment without touching source files:

- RAW_TRACKS_PATH — path to the raw dataset
- PROCESSED_MUSIC_PATH — path to cleaned data
- MOOD_MODEL_PATH — path to trained model
- LABEL_ENCODER_PATH — path to label encoder
- FEATURE_COLUMNS — list of 9 audio features
- TARGET_COLUMN — mood
- TOP_N_RECOMMENDATIONS — default number of recommendations

- ## Utility Modules

utils.py provides shared helper functions used across the project:

- get_logger(name) — creates a logger that writes to both console and a dated log file in logs/
- save_model(model, path) — serialises any sklearn object to a .pkl file
- load_model(path) — loads a .pkl file with a clear FileNotFoundError if missing
- ensure_dir(path) — creates a directory if it does not exist
- file_exists(path) — returns True if a file exists at the given path

- ## Model Performance and Evaluation

The model was evaluated on a held-out test set of 22,710 songs (20% of the full dataset). The following performance metrics were recorded:

Metric	Value
Training Accuracy	100.0%
Test Accuracy	99.1%
Precision (avg)	>98% across all classes
Recall (avg)	>98% across all classes
F1 Score (avg)	>98% across all classes
Training Samples	90,839 songs
Test Samples	22,710 songs
Number of Features	9 audio features

Number of Classes	4 moods

Note: A training accuracy of 100% indicates that the model has fully memorised the training data, which is characteristic of unpruned Random Forests. The gap between train (100%) and test (99.1%) accuracy is small, suggesting the model generalises well despite the perfect training fit. Future work could explore max_depth or min_samples_split constraints to reduce this gap further.

• ***Testing***

The project includes a comprehensive pytest test suite covering all three core modules. Tests are located in the tests/ directory and are run via:

```
pytest tests/ -v
```

Test File	Tests	What is Verified
test_preprocessing.py	4 tests	Feature columns presence, target column, no nulls, processed file exists
test_model.py	5 tests	Model file exists, predict works, predict_proba works, label encoder classes, feature count
test_recommendation.py	5 tests	Returns results, correct mood filter, count limit, invalid mood handling, track_name column

All 14 tests pass successfully. The pytest.ini configuration enforces strict marker usage and verbose output by default, with custom markers for slow and integration tests.

• ***Results***

The system was tested end-to-end with the following sample CLI interaction:

```
🎵 Mood Based Music Recommendation System
----------------------------------------
Available moods: Happy, Sad, Angry, Calm
Enter your mood: HAPPY

🎵 Top 10 songs for your mood (Happy):

                        track_name                        artists track_genre
                    Maldito Sudaca                Los Prisioneros         ska
                     Vamos Hacerlo                          Dvice      reggae
                        Starstruck                  Years & Years         edm
                           Pangeia     BaianaSystem;Mateus Aleluia    afrobeat
                   Speedy Gonzales                      Pat Boone rock-n-roll
        Quando o Grande Amor Se Faz                 Bonde do Forró      pagode
          The Boys Are Back In Town                     Thin Lizzy       metal
Popotitos/La Plaga - En Vivo/Medley Enrique Guzman;Alejandra Guzman rock-n-roll
                          Eightfold                   Violent Soho      garage
                        Pensamentos                            SNJ      brazil
PS C:\Users\piyus\OneDrive\Desktop\aiml project>
```

The recommendations span multiple genres, which is expected given that mood is determined by acoustic properties rather than genre classification. A happy track can belong to blues, k-pop, reggae, or techno — the system correctly captures this cross-genre mood consistency.

• Challenges and Solutions

Challenge	Solution
No mood labels in raw dataset	Derived mood labels from audio feature heuristics based on music psychology thresholds
config module name conflict with PyPI package	Used sys.path manipulation to ensure local config.py is imported over the PyPI config package
Large model files exceeding GitHub 100 MB limit	Added model/*.pkl to .gitignore; models are regenerated locally via python main.py train
top-k argument not passed through CLI to recommendation	Fixed main.py to include --top-k argument and pass it to run_recommendation()
Another contributor's commits appearing in repo history	Reinitialised Git history locally and force pushed to a clean new repository

• Future Enhancements

- Integrate with the Spotify Web API to fetch real-time audio features for any track.
- Replace the rule-based mood labelling with a human-annotated dataset for more nuanced labels.
- Experiment with gradient boosting models (XGBoost, LightGBM) to compare performance.
- Add a web-based frontend using Flask or Streamlit for a more accessible user interface.
- Introduce user profiles to personalise recommendations based on listening history.

- Extend mood categories beyond four to capture a richer emotional spectrum.
- Add model versioning and experiment tracking with MLflow or Weights & Biases.

Conclusion

This project successfully demonstrates the application of supervised machine learning to the problem of mood-based music recommendation. A Random Forest classifier trained on nine Spotify audio features achieves 99.1% test accuracy across four mood categories — Happy, Sad, Angry, and Calm — on a dataset of over 113,000 songs.

Beyond the machine learning component, the project applies sound software engineering practices: centralised configuration, shared utility functions, a complete automated test suite, and a clean CLI interface. The codebase is modular and maintainable, with clear separation between data processing, model training, and recommendation logic.

The system delivers fast, diverse, cross-genre recommendations that are grounded in the acoustic properties of music rather than listening history — making it a genuinely mood-aware recommendation engine.

References

- Spotify Web API Audio Features Documentation — https://developer.spotify.com/documentation/web-api
- scikit-learn: Machine Learning in Python — Pedregosa et al., JMLR 12, pp. 2825-2830, 2011
- Spotify Tracks Dataset — Kaggle, accessed March 2026

- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
- Russell, J.A. (1980). A circumplex model of affect. Journal of Personality and Social Psychology, 39(6).
- Project Repository — https://github.com/iampiyushsingh/Vityarthi_Project_25BAI11043